

Testing, Debugging, and Deployment

Apply techniques and tools for testing and debugging Lightning Web Components, Aura Components, and Visualforce controllers, Visualforce controller extensions, and JavaScript.

Develop your career with [Salesforce Training and Certification Preparation](#)

Table of Contents

-  [Testing Visualforce Controllers and Extensions](#)
-  [Testing Lightning Web Components](#)
-  [Viewing Debug Information](#)
-  [Testing With UTAM](#)
-  [Case Sensitivity in JavaScript](#)
-  [Implicit Coercion in JavaScript](#)
-  [Truthy and Falsy in JavaScript](#)
-  [Mutable Assignment in JavaScript](#)



Table of Contents

-  [Variable Scope in JavaScript](#)
-  [Object Notation in JavaScript](#)
-  [Try...Catch Statement in JavaScript](#)



After studying this topic, you should be able to:

-  Describe the considerations for creating unit tests for Visualforce controllers and controller extensions.
-  Identify the steps for setting up unit tests for a Visualforce controller or controller extension.
-  Describe how to test Aura components and Lightning web components using available tools.
-  Identify important JavaScript characteristics and the considerations when writing or debugging.



Introduction

Since **Visualforce controllers** and **controller extensions** are Apex classes, their tests should have code coverage that meets requirements. When testing controllers and extensions, the unit test involves writing code that simulates user input from the Visualforce page. The **Jest** testing framework is recommended for testing Lightning web components. For Aura components, several tools are available in addition to Jest such as **UTAM**, **Jasmine**, **Mocha**, **Selenium**, and **WebdriverIO**.

This topic covers techniques and tools that are used to test **Visualforce code** and **custom Lightning components**. In addition, certain **JavaScript characteristics** are described in order to understand its behavior when either writing or debugging JavaScript code.



Testing Controllers & Controller Extensions

PageReference

In a test method, the PageReference for the Visualforce page is created.

Action Methods

Controller's action methods can be called for testing user action.

URL Parameters

URL parameter's name and value can be passed to the controller.

Test.setCurrentPage()

The PageReference variable is passed to Test.setCurrentPage().

Instantiation

The controller and controller extension for the page are instantiated.



Testing Lightning Components

Jest Framework

The Jest framework can be utilized to test the functionality of Lightning web components in an org. Jest tests can be run in the command line or the IDE.

Test Suites

A 'describe' block can be defined in a test file to create a test suite that contains multiple unit tests for a particular component.

Testing Aura components

For Aura components, the following testing frameworks can be used: Jest, UTAM, Jasmine, Mocha, Selenium, and WebdriverIO.

Unit Tests

An 'it' or 'test' block can be defined to create a single functional unit test that tests a particular behavior in an Aura or Lightning web component.



Testing Visualforce Controllers and Extensions

Table of Contents



Testing Visualforce Controllers and Controller Extensions

Like all **Apex code**, custom controllers and controller extensions should be **covered** by **unit tests**.



UNIT TESTS

Unit tests for Visualforce **controllers** and **extensions**:



CLASS METHODS

Unit tests are class methods that are written to test the **business logic** and verify that code functions **as expected**.



NO ARGUMENTS

Unit tests **do not accept** any arguments or method parameters.



NO COMMIT

Unit tests **do not** commit any data to the **database**.



ANNOTATION

Unit test **methods** are annotated with the **@isTest** annotation.

Testing Visualforce Controllers and Controller Extensions



SETTING UP

The high-level code steps for **setting up a test** of a Visualforce controller or extension:



APEX CLASS

Create a **test class** with a **test method** using the `@isTest` annotation.



PAGE REFERENCE

Create a `PageReference` to the **Visualforce page**.



CURRENT PAGE

Pass the `PageReference` instance to **`Test.setCurrentPage()`** to set the current `PageReference`.



CONTROLLER

Instantiate the controller for the **Visualforce page**.



EXTENSION

If needed, **instantiate** an extension for the page by passing the **controller variable** as a parameter to the extension **constructor**.



PARAMETERS

If needed, pass **URL parameters** to the controller using the following **format**:
`ApexPages.currentPage().getParameters().put('paramName', 'paramValue');`

Testing Visualforce Controllers and Controller Extensions



SIMULATE INPUT

If needed, use the controller's **setter methods** to simulate user input for data fields.



SIMULATE CLICK

Call the controller's **action methods** to simulate a user clicking a button.



ASSERT RESULTS

Assertion methods are used to **validate** or **invalidate** business logic output.



CLASS NAME

A test class **cannot be named Test**. Doing this generates the following **error message**:
"Method does not exist or incorrect signature: Test.setCurrentPage(System.PageReference)"



PAGE REDIRECT

If the controller redirects to another page based on user input, the resulting **page name** can be checked by calling the **getUrl() method** of the resulting **PageReference** variable.

Example of Testing a Visualforce Controller

The image below shows a Visualforce page titled, 'AccPage,' that allows **receiving and saving account**.

```
<apex:page controller="CustomController" tabStyle="Account">
<apex:form>
  <apex:pageBlock title="Account Page">
    <apex:pageBlockSection>
      <apex:inputField value="{!account.Name}"/>
      <apex:inputField value="{!account.Rating}"/>
      <apex:inputField value="{!account.Type}"/>
      <apex:inputField value="{!account.Industry}"/>
      <apex:commandButton action="{!save}" value="save"/>
    </apex:pageBlockSection>
  </apex:pageBlock>
</apex:form>
</apex:page>
```

Example of Testing a Visualforce Controller

The image below shows a **custom controller** for the Visualforce page titled, 'AccPage.'

```
public class CustomController {  
  
    public final Account account;  
  
    public CustomController() {  
        account = [SELECT Id, Name, Rating, Type, Industry FROM Account  
                   WHERE Id = :ApexPages.currentPage().getParameters().get('id')];  
    }  
  
    public Account getAccount() {  
        return account;  
    }  
  
    public PageReference save() {  
        update account;  
        return null;  
    }  
}
```

Example of Testing a Visualforce Controller

The screenshot below illustrates the **testing** of the custom controller.

```
@isTest
public class CustomControllerTest {
    @isTest
    public static void testMethod1() {
        Account a = new Account(); // A test account is created with the required field values
        a.Name = 'Test';
        insert a;
        a.Rating = 'Hot';
        a.Type = 'Customer';
        a.Industry = 'Healthcare';

        PageReference testPage = Page.AccPage; // A page reference for the Visualforce page is created
        Test.setCurrentPage(testPage); // A method is used to set the current page reference
        ApexPages.currentPage().getParameters().put('id', a.Id); // Query parameters can be set and used in the test
        CustomController cc = new CustomController(); // The custom controller is instantiated
        PageReference pageRef = cc.save(); // The methods in the custom controller are called
        Account b = cc.getAccount();
        // Check if controller worked on the same record System.assertEquals(b.Id, a.Id);
    }
}
```


Example of Testing a Visualforce Controller Extension

Illustrated below is a Visualforce Page titled, 'CustomPage,' that provides **additional details about an account through a controller extension.**

```
<apex:page standardController="Account" extensions="ControllerExtension">
  <apex:form>
    <apex:pageBlock>
      <apex:pageBlockSection>
        <p>Account Name: {!account.Name}</p>
        <p><apex:outputText value="{!details}" /></p>
        <apex:outputField rendered="false" value="{!account.Type}" />
        <apex:outputField rendered="false" value="{!account.Industry}" />
      </apex:pageBlockSection>
    </apex:pageBlock>
  </apex:form>
</apex:page>
```


Example of Testing a Visualforce Controller Extension

Depicted below is a **controller extension** for the Visualforce page titled, 'CustomPage.'

```
public class ControllerExtension {  
  
    public final Account account;  
  
    public ControllerExtension(ApexPages.StandardController stdController) {  
        this.account = (Account)stdController.getRecord();  
    }  
  
    public String getDetails() {  
        return 'Account Type and Industry are ' + account.Type + ' and ' + account.Industry;  
    }  
}
```

Example of Testing a Visualforce Controller Extension

The image below illustrates the **testing** the controller extension.

```
//Test class for the controller extension named 'ControllerExtension'
@Test
public class ControllerExtensionTest {

    @Test
    public static void testMethod1() {

        //A test account is created with the required field values
        Account a = new Account();
        a.Name = 'Test';
        a.Type = 'Customer';
        a.Industry = 'Healthcare';
        insert a;

        // A page reference for the Visualforce page is created
        PageReference testPage = Page.CustomPage;

        //A method is used to set the current page reference
        Test.setCurrentPage(testPage);
    }
}
```

Continued...

Example of Testing a Visualforce Controller Extension

See below for the continuation of the **testing** the controller extension.

```
// Query parameters can be set and used in the test
ApexPages.currentPage().getParameters().put( 'id', a.id );

// The standard controller for the page is instantiated
ApexPages.StandardController sc = new ApexPages.StandardController(a);

// ControllerExtension is instantiated by passing the standard controller variable
ControllerExtension ce = new ControllerExtension(sc);

// The getDetails() method of the ControllerExtension class is called
String str = ce.getDetails();

// A System.assert statement is used to verify that the getDetails() method returns the correct string
System.assertEquals('Account Type and Industry are Customer and Healthcare', str);
}
}
```

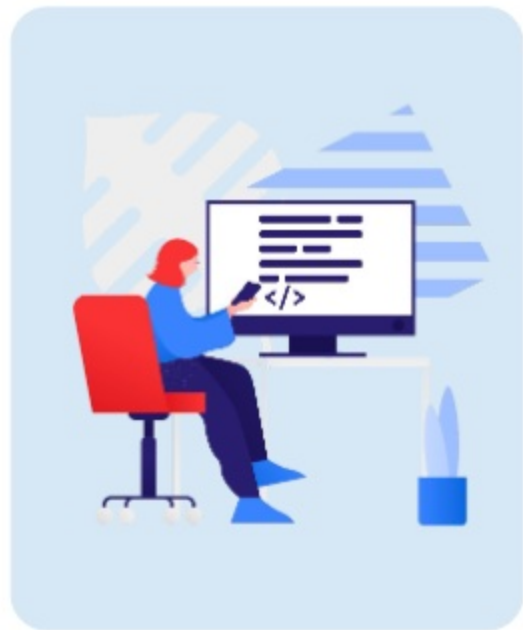
Testing Lightning Web Components

Table of Contents



Testing Lightning Web Components

The **Jest framework** can be used to write **unit tests** for Lightning web components.



❖ RUN TESTS

Jest tests can be run in the **command line** or the **IDE**.

❖ TEST CASES

Jest tests can be written to test a **component in isolation**, its **public API**, **basic user interaction**, **DOM output verification**, **firing of events**, etc.

Testing Lightning Web Components

The following **steps** can be **utilized** for using Jest:



STEP 1: SALESFORCE DX

Create a **Salesforce DX project** and install **sfdx-lwc-jest** and its dependencies (after installing **Node.js** and **npm**). This Salesforce CLI command can be used: **sf force lightning lwc test setup**



STEP 2: TEST FILES

Write Jest tests in local **JavaScript files**. The Salesforce CLI command, **"sf force lightning lwc test create"** can be used to create a **test directory** named **"__tests__"** and a boilerplate **test file** within the directory. It is recommended that test files end in **".test.js"**.



STEP 3: TEST COMMAND

Run Jest tests from **VS Code** or the **command line** using the command: **"sf force lightning lwc test run"**

Writing Jest Tests

A **Jest** test should be designed to test the **behavior** of a **single component** in isolation. The following are some of the **most important** aspects of a Jest test:

◆ IMPORT COMPONENT

Each test file must import both the **component** to **test** and the **createElement** method from **'lwc'**.

◆ DESCRIBE BLOCK

A describe block should be **defined** to create a **test suite** within the test file.



Writing Jest Tests

◆ DOM RESETS

Since a single instance of `jsdom`, which is used to **simulate** a DOM environment in a browser, is shared across all the **test cases** in a test file, the DOM should be **reset** at the end of each unit test using the `afterEach()` method.

◆ TEST BLOCK

An **it** or **test** block should be defined to create a single functional **unit test**. Its **first argument** is a short description of the test and the **second argument** is a function containing the test. The **expect** statement can be used with a matcher like **toBe** for an assertion in the unit test.



Writing Jest Tests

◆ CREATE INSTANCE

Inside the unit test, the imported `createElement` method can be used to create an `instance` of the component for testing. It should be `appended` to the body of the document using the `appendChild()` method.



Writing a Jest Test

The following code snippet shows an example of the **LWC markup** and **Jest unit test**.

```
import { createElement } from 'lwc';
import LwcTestDemo from 'c/lwcTestDemo';
describe('c-lwc-test-demo', () => {
  afterEach(() => {
    // The DOM should be reset after each test.
    while (document.body.firstChild) {
      document.body.removeChild(document.body.firstChild);
    }
  });
  // The 'it' or 'test' keyword can be used to create a unit test block.
  it('displays heading, name, and email', () => {
    const element = createElement('c-lwc-test-demo', { is: LwcTestDemo });
    document.body.appendChild(element);
    const heading = element.shadowRoot.querySelector('h1').textContent;
    const name = element.shadowRoot.querySelector('.name').textContent;
    const email = element.shadowRoot.querySelector('.email').textContent;
    expect(heading).toBe('Customer Details'); // The 'expect' and 'toBe' keywords can be used for assertions.
    expect(name).toBe('Name: Ashley James');
    expect(email).toBe('Email: ashley@united.com');
  });
});
```

```
<!-- lwcTestDemo.html -->
<template>
  <h1>Customer Details</h1>
  <p class="name">Name: Ashley James</p>
  <p class="email">Email: ashley@united.com</p>
</template>
```

This example shows a Jest test for a Lightning web component named *lwcTestDemo*. The test file has a test suite containing a single unit test that verifies whether the expected heading, name, and email are displayed in the component.

Running Jest Tests

The `npm run test:unit` command can be used in the terminal to run Jest tests. The example below shows the use of the command, which results in running all the test suites.

```
PS C:\dev\test-lwc> npm run test:unit

> salesforce-app@1.0.0 test:unit C:\dev\test-lwc
> sfdx-lwc-jest

PASS force-app/main/default/lwc/lwcTestDemo/__tests__/lwcTestDemo.test.js
  c-lwc-test-demo
    ✓ displays heading, name, and email (34ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        3.057s
Ran all test suites.
```

Considerations for Testing Lightning Web Components

There are certain **considerations** for testing Lightning web components using the **Jest** framework:

SHADOWROOT



When creating a unit test, the **shadowRoot** property of the element under test should be used to access the shadow tree. It can be used to select a particular element. For example, **`element.shadowRoot.querySelector('h1')`** can be used to select the `<h1>` element.

USE PROMISES



Asynchronous DOM updates such as property change can be tested by using a **Promise** in the unit test.

Considerations for Testing Lightning Web Components



WIRE SERVICE

To test the wire service, the [@salesforce/sfdx-lwc-jest test utility](#) can be utilized.



JSON FILE

A JSON file can be [created](#) to define the Salesforce data [expected](#) by a component.



WATCH MODE

The “`npm run test:unit:watch`” command can be used to [run tests continuously](#) during development.

Viewing Debug Information

Table of Contents



Viewing Debug Information

This example uses the following Lightning web component, which **displays contact information**.

```
// contact.js
import { LightningElement } from 'lwc';
import getContacts from '@salesforce/apex/ContactController.getAll';

export default class Contact extends LightningElement {

    contacts;
    contact;
    name;
    email;
    phone;
    title;

    connectedCallback() {
        getContacts().then(results => this.contacts = results);
    }

    ...
}
```

When a user enters a name in an input field that matches an existing contact, the **contact**, **name**, **email**, **phone**, and **title** properties are updated to render details of that contact.

This component queries contacts and stores the results in a **contacts** property upon loading.

Viewing Debug Information

In the **Elements** tab in Chrome DevTools, the **component should be selected**. Then, in the **Console** tab, **\$0** is entered to display the debug information of the Lightning web component.

The screenshot displays the Chrome DevTools interface. On the left, the application view shows a 'Contact Details' form with fields for Name, Email, Title, and Phone, and a 'Get Contact Details' button. The 'Elements' tab on the right shows the component tree, with the '<c-contact>' element selected and highlighted with a red box. Below the component tree, the console shows the command '> \$0' entered, with the output 'c-contact' and 'LWC: Contact' displayed. A yellow callout box notes that '@wired properties are also accessible in this debug information.' An arrow points from the console output to a detailed view of the component's properties, which includes fields like 'Email', 'Id', 'Name', 'Phone', 'Title', and 'contacts'.

Note that @wired properties are also accessible in this debug information.

```
<flexipage-component2 data-component-
selection-name="c_contact" slot="reg
component2_component2-host>
  <slot flexipage-component2_componer
    <c-contact> == $0
      <lightning-card>
        <article class="slds-card" pa
          <div class="slds-card_heade
            <header class="slds-media
... v.wrapper div.grouping.grouping1 div.col
  Console What's New Issues
  top Filter
  > $0
  < > c-contact
    LWC: > Contact
  >
```

```
className: (...)
contact: Object
  Email: "abailey@dennys.com"
  Id: "0032v00002qoFcxAEE"
  Name: "Adolfo Bailey"
  Phone: "(864) 597-8000"
  Title: "Account Manager"
  > [[Prototype]]: Object
  > contacts: Array(532)
  contentEditable: (...)
  dataset: (...)
  dir: (...)
  draggable: (...)
  dropzone: (...)
  email: "abailey@dennys.com"
  hidden: (...)
  id: (...)
  inputMode: /
```

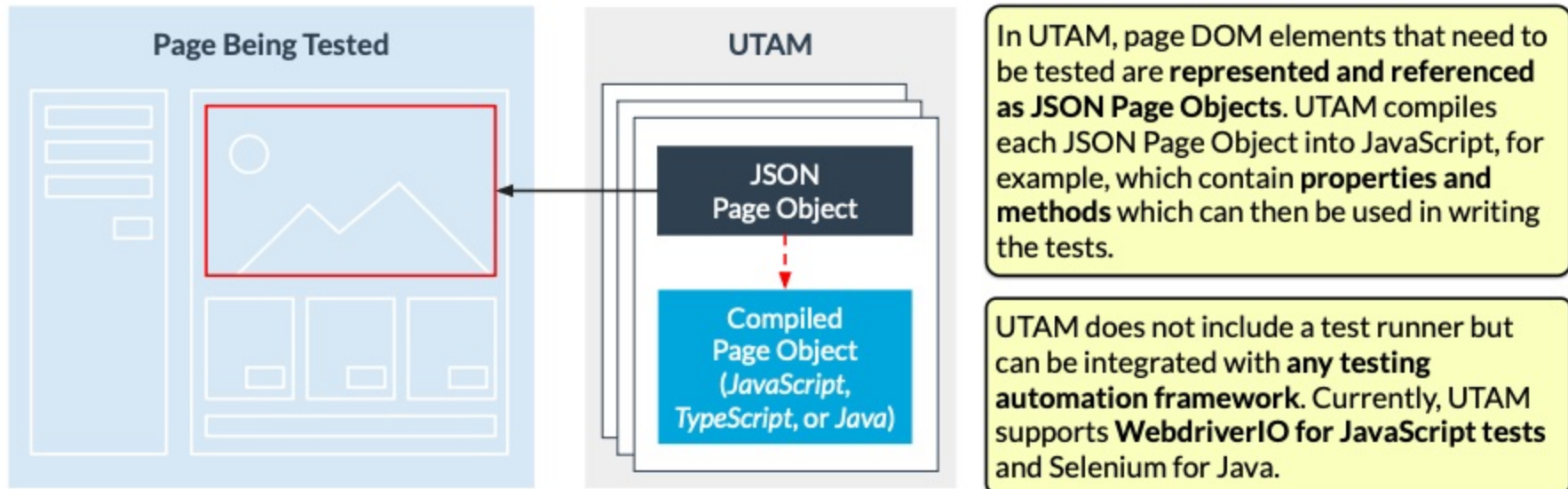
Testing With UTAM

Table of Contents



UI Test Automation Model (UTAM)

UTAM is an open-source tool built by Salesforce which can be used to **test any web page** and is ideally suited for writing **JavaScript or Java UI tests for Lightning web components**.



UI Test Automation Model (UTAM)

The following shows a simplified example of how UTAM is used in **testing a user interface**. A **UTAM Generator** npm package can also be used to automatically generate JSON page objects from HTML files.

```
// Sample JSON Page Object created based on HTML below
{
  "root": true,
  "selector": {
    "css": "body"
  },
  "elements": [
    {
      "selector": {
        "css": ".color"
      },
      "name": "color",
      "public": true
    }
  ]
}
```

The diagram shows a yellow box labeled "CSS selector" with an arrow pointing to the "css" property of the "selector" object in the JSON. Another yellow box labeled "Generates a getColor() method" has an arrow pointing to the "name" property of the "elements" array item in the JSON.

Compiled into

```
// Sample code snippet of compiled Page Object (JavaScript)
// Error codes/messages are reported if there are compilation errors
...
async getColor() { // generated method based on JSON Page Object
  const driver = this.driver;
  const root = await this.getRootElement();
  const BaseUtamElement = _createUtamMixinCtor();
  let element = await _utam_get_color(driver, root);
  element = new BaseUtamElement(driver, element);
  return element;
}
...
```

Sample JavaScript test

```
...
// load the page object
const colorCheckRoot = await utam.load(ColorCheckRoot);
assert.ok(colorCheckRoot instanceof ColorCheckRoot);

// call UTAM-generated method to retrieve the span element
const color = await colorCheckRoot.getColor();
assert.ok(color); // verify if the element exists on the page

const colorName = await color.getText();
assert.strictEqual(colorName, 'blue'); // verify if text matches
...
```

```
<!-- Rendered HTML of page that is being tested
<body>
  <div class="main">
    <h3>Your favorite color is
      <span class="color">blue</span>
    </h3>
  </div>
</body>
```

UTAM Generator

A **generator playground** can be used to generate **UTAM page objects** from HTML code. The generator can be found in utam.dev.

The screenshot displays the UTAM Generator playground interface. At the top, the navigation bar includes the logo, "UI Test Automation Model", and links for Downloads, Grammar, Guide, Tutorials, Salesforce, Tools, and a search bar. A note states "Safari browser is currently not supported".

The main interface is divided into three panels:

- Config examples:** A dropdown menu currently set to "empty". Below it is a link for "Generator Config (optional) [documentation](#)".
- HTML source code:** A text area containing the following HTML code:

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4
5 <h1>My First Heading</h1>
6 <p>My first paragraph.</p>
7 |
8 </body>
9 </html>
```
- Generated UTAM JSON:** A text area showing the corresponding UTAM JSON output:

```
1 {
2   "description": {
3     "author": "UTAM generator",
4     "text": [
5       "Page Object: page object name",
6       "Root element: html"
7     ]
8   },
9   "root": true,
10  "selector": {
11    "css": "html"
12  },
13  "elements": [
14    {
15      "public": true,
16      "name": "h1Content",
17      "type": [],
18      "selector": {
19        "css": "h1"
20      }
21    },
22    {
23      "public": true,
24      "name": "pContent",
25      "type": [],
26      "selector": {
27        "css": "p"
28      }
29    }
30  ]
31 }
```

Two yellow callout boxes with black borders are positioned in the center:

- A box labeled "Source HTML code" with an arrow pointing to the HTML source code panel.
- A box labeled "Generated UTAM page objects" with an arrow pointing to the Generated UTAM JSON panel.

Case Sensitivity in JavaScript

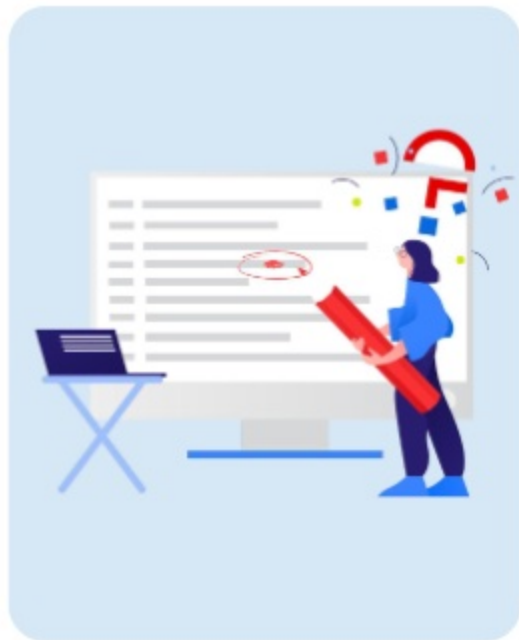
Table of Contents



Case Sensitivity

Unlike Apex or SOQL, JavaScript is **case-sensitive**.

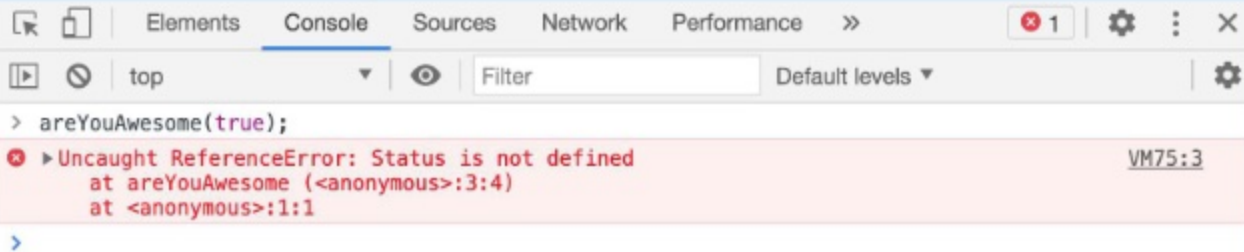
- ❖ **SCENARIO**
- An Apex developer who recently started learning JavaScript has written a sample JavaScript function that accepts a parameter and logs a string in the browser console based on the passed value. However, when the function was called, an error was encountered.



Case Sensitivity

The JavaScript **function** below when run **does not** produce the expected result.

```
function areYouAwesome(status) {  
  
  if (Status == true) {  
    console.log('Stay awesome.');  } else {  
    console.log('You are still awesome.');  }  
}  
areYouAwesome(true);
```



Case Sensitivity



SOLUTION

Unlike Apex or SOQL, JavaScript is **case-sensitive**. To fix the error, the variable name in the *if* statement should be written as "**status**", instead of "**Status**", in order to match the case in the function parameter definition.

Therefore, the code should be changed to the following: **if (status == true)**. Or, alternatively the parameter in the function definition can be updated instead to match the case.

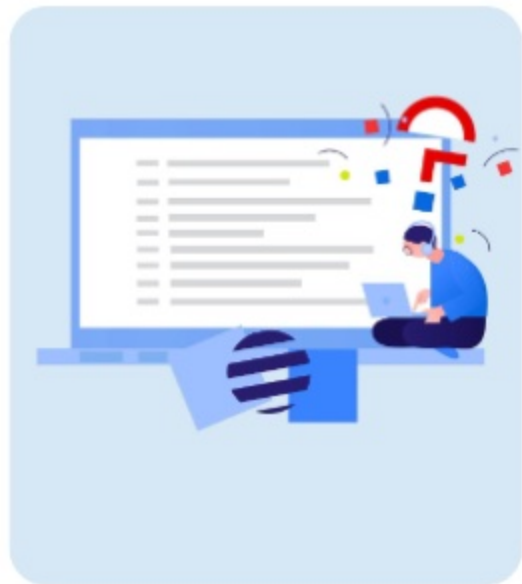
Implicit Coercion in JavaScript

Table of Contents



Implicit Type Coercion

JavaScript convert **invalid types** to **valid types**. The `===` and `!==` operators should be used to compare both the **value** and **data type**.



❖ SCENARIO

A developer is performing a series of operations and comparisons to determine how JavaScript variables behave based on the operator and operands used in each expression. However, the results do not seem to make sense. The developer would like to understand this behavior in order to write code that produces expected results.

Implicit Type Coercion

The following examples illustrate seemingly unexpected results, depending on the **data type combination** and **operator** used in an expression.

```
let variable1 = 10 * "30";
console.log(variable1); // 300
console.log(typeof variable1); // number
let variable2 = 10 + "30";
console.log(variable2); // "1030"
console.log(typeof variable2); // string
console.log(false == ""); // returns true
console.log(false == "0"); // returns true
console.log("" == "0"); // returns false
console.log([0] == 0); // returns true
```



Console What's New

top Filter Default levels

```
> let variable1 = 10 * "30";
console.log(variable1); // 300
console.log(typeof variable1); // number
let variable2 = 10 + "30";
console.log(variable2); // "1030"
console.log(typeof variable2); // string
console.log(false == ""); // returns true
console.log(false == "0"); // returns true
console.log("" == "0"); // returns false
console.log([0] == 0); // returns true
```

300	VM88:2
number	VM88:3
1030	VM88:5
string	VM88:6
true	VM88:7
true	VM88:8
false	VM88:9
true	VM88:10

Implicit Type Coercion



SOLUTION

This behavior is called **implicit type coercion**. In a situation where operands of **different** data types are used in a **single** expression, JavaScript automatically converts an **"invalid"** data type to a **"valid"** type. Although there is a system to JavaScript evaluation of the data types, it is best not to use implicit type coercion, to avoid confusion.

To determine whether two variables are equal, it is best practice to use the **strict comparison (===) operator**. It will only return **true** if both the variable **value** and **data type** are the same. The **!==** operator can be used to return true if both the value and the data type are **not** the same.

Truthy and Falsy in JavaScript

Table of Contents



Truthy and Falsy

A **truthy** is any value that evaluates to **true** in a Boolean context. On the other hand, a **falsy** value evaluates to a Boolean **false** value.



TRUTHY

Any other value (except **false**) is coerced as **true** (except true).



FALSY

When an **expression** expects a Boolean value, the following **non-Boolean** values always return **false**:

- ❖ 0, -0, or 0n (**BigInt**)
- ❖ "" or " (**empty string**)
- ❖ null
- ❖ undefined
- ❖ NaN (**not a number**)

Truthy and Falsy

Conditional statements do not require actual **Boolean values** and operate based on the **truthness** or **falsehood** of a value of any data type.

```
var a = null; // falsy value
if (a) {
  console.log('branch 1');
} else {
  console.log('branch 2');
}

var b = [0]; // truthy value
if (b) {
  console.log('branch 3');
} else {
  console.log('branch 4');
}
```



```
> var a = null; // falsy
  if (a) {
    console.log('branch 1');
  } else {
    console.log('branch 2');
  }

  var b = [0]; // truthy
  if (b) {
    console.log('branch 3');
  } else {
    console.log('branch 4');
  }

branch 2
branch 3
< undefined
> |
```

Any value that is not falsy will always return true when evaluated in a condition. The **Boolean function can be used to determine a falsy or truthy value such as `Boolean('false');` // truthy**

Mutable Assignment in JavaScript

Table of Contents



Mutable Assignment

In JavaScript, there are **three ways** to declare a variable: **var**, **let**, and **const**.

KEYWORD	MUTABLE	SCOPE	DESCRIPTION
var	Yes	function	Variable is mutable , meaning it can be reassigned values as many times as needed. It lives within the function scope.
let	Yes	block	Variable is mutable , meaning it can be reassigned values as many times as needed. It lives within the block scope.
const	No	block	Variable is not mutable , meaning it can be assigned a value only once . Assigning a value again will throw an error. It lives within the block scope.

Mutable Assignment

◆ SCENARIO

The code snippet in the next slide contains multiple variable assignments. When the code is executed, the last line (type variable reassignment) throws an “*Assignment to constant variable*” error. The developer needs to determine why this happens for the type variable but not for the others.



Mutable Assignment

The JavaScript **variables** below are declared using **var**, **let**, and **const** to show how each variable behaves in a **reassignment**.

```
// primitive value assignments
var color = "silver";
let topSpeed = 300;
const type = "sedan";

// reassignment
color = "black";
topSpeed = 320;
type = "hatchback"; // oops!
```



The screenshot shows a browser console window with the following content:

```
> // primitive value assignments
var color = "silver";
let topSpeed = 300;
const type = "sedan";

// reassignment
color = "black";
topSpeed = 320;
type = "hatchback"; // oops!
```

An error message is displayed in a red box:

```
Uncaught TypeError: Assignment to constant variable.
    at <anonymous>:9:6
```

On the right side of the console, a yellow callout box contains the text:

The **type** variable is a constant which is assigned a value during its declaration. Reassigning a value to it is not allowed.

Mutable Assignment

JavaScript **objects** are **mutable**. An object that is declared with **const** possesses the following **characteristics**:



MUTABLE

Properties of a JavaScript constant **object** are mutable.



EXTENDIBLE

Properties or **methods** can still be **added** after declaration or initialization.



REASSIGN

However, reassignment of the constant variable **throws** an **error**.

Mutable Assignment

The following illustrates how properties of a **constant object** are mutable, and that **reassignment** of the variable itself causes an **error**.

```
const x = {};  
  
// add a new property  
x.skill = 'talking';  
  
// add a new function  
x.talk = function(){  
  console.log('hello');  
};  
x.talk(); // prints "hello"  
  
// but...  
  
x = {}; // error!  
x = true; // error!
```

A screenshot of a web browser's developer console. The console shows the execution of the code from the left panel. It includes comments like '// add a new property', '// add a new function', and '// prints "hello"'. At the bottom, it shows two lines of code that cause errors: 'x = {};' and 'x = true;'. Below these, a red error message is displayed: 'Uncaught SyntaxError: Identifier 'x' has already been declared'. The error message is followed by a link 'VM338:1'.

```
> const x = {};  
  
// add a new property  
x.skill = 'talking';  
  
// add a new function  
x.talk = function(){  
  console.log('hello');  
};  
x.talk(); // prints "hello"  
  
// but...  
  
x = {}; // error!  
x = true; // error!  
  
✖ Uncaught SyntaxError: Identifier 'x' has already been declared VM338:1  
>
```

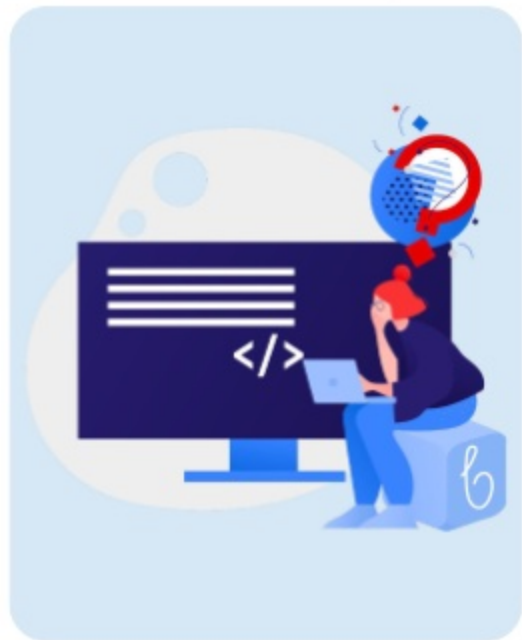
Variable Scope in JavaScript

Table of Contents



Function Scope vs. Block Scope

The **keyword** (*var*, *let*, or *const*) used to declare a variable impacts the **variable scope**.



❖ SCENARIO

When the code snippet displayed in the next slide is executed, the developer expects to see a string printed in the browser console that states, *"I am 16 years old!"*. However, after running the code, the output was not exactly what was expected.

Function Scope vs. Block Scope

When executed, the code below **returns** the following **output**:

```
let howOldAmI = function() {  
  
  var num = 16;  
  
  for (var num = 0; num < 3; num++){  
    // do something cool  
  }  
  
  console.log('I am ' + num + ' years old!');  
}  
  
howOldAmI();
```

A screenshot of a browser console window. The title bar says "Console" and "What's New". There are icons for running, stopping, and refreshing, along with a "top" button and a "Filter" input. Below the toolbar, the code from the previous block is pasted. The output of the console.log statement is "I am 3 years old!", which is highlighted with a red box. The console also shows "undefined" and a "VM343:9" reference.

```
> let howOldAmI = function() {  
  var num = 16;  
  for (var num = 0; num < 3; num++){  
    // do something cool  
  }  
  console.log('I am ' + num + ' years old!');  
}  
howOldAmI();  
I am 3 years old!  
< undefined  
>
```

Function Scope vs. Block Scope



❖ SOLUTION

Running the code above prints *"I am 3 years old!"*, and this is due to the **declaration** of the "num" variable using the **var** keyword in the *for* loop definition. When the variable is declared using this keyword, its scope is **hoisted** and moved to the **function** scope where the former "num" variable is declared. Hence, the former "num" variable is **overwritten** in each *for* loop iteration. Improper use of "var" causes confusion and accidental overwrites.

Please refer to the table in the Mutable Assignment section above.

Function Scope vs. Block Scope

Using the **let** keyword in the **for loop** declaration produces a different result since the **scope** of the **let** variable is **within** the block where it is defined, and the variable is **inaccessible outside** of the block.

```
let howOldAmI = function() {  
  var num = 16;  
  
  for (let num = 0; num < 3; num++){  
    // do something cool  
  }  
  
  console.log('I am ' + num + ' years old!');  
}  
  
howOldAmI();
```



```
> let howOldAmI = function() {  
  var num = 16;  
  
  for (let num = 0; num < 3; num++){  
    // do something cool  
  }  
  
  console.log('I am ' + num + ' years old!');  
}  
  
howOldAmI();  
I am 16 years old!  
< undefined  
> |
```

It's important to note that **const** variables also operate within the **block** scope.

Object Notation in JavaScript

Table of Contents

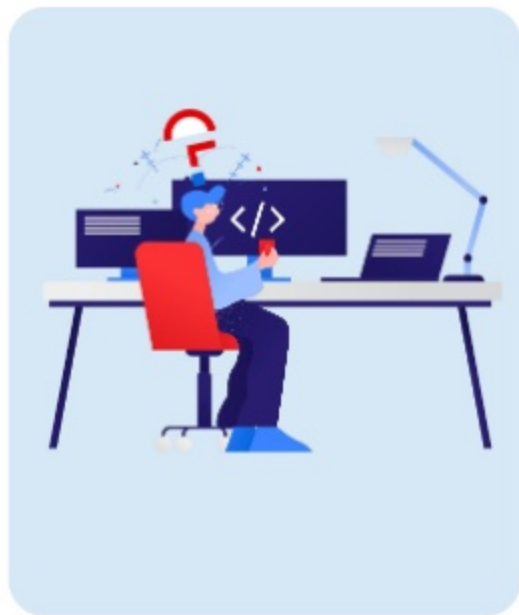


Object Notation

Objects can be created using an **object literal** or an **object constructor notation**.

◆ SCENARIO

A developer has written JavaScript code to create an object called 'Vehicle' and would like to determine any limitations of the object when declaring the variable in this manner.



Object Notation

The code below shows a JavaScript **object variable** that is declared using the **object literal notation**.

```
let Vehicle = {  
  topSpeed: 300,  
  state: '',  
  move: function(status) {  
    if (status === true) {  
      this.state = 'moving';  
    } else {  
      this.state = 'stopped';  
    }  
  }  
}  
  
Vehicle.move(true);  
console.log(Vehicle.state); // prints "moving"
```

Object Notation



SOLUTION

In JavaScript, there are **two methods** that can be used to **create** an object:

- ❖ Object Literal Notation
- ❖ Object Constructor Notation

The Vehicle object above is an example of a variable declared using the **Object Literal Notation**. The **limitation** for objects declared using this type is that there can only be **one instance** of the object. In addition, Object Literal Notation is **not** a recommended practice in **modern** JavaScript. **Object literals** may be used for encapsulating **data**, but defining **functions** within it should be avoided.

Object Notation

If it is **required** for an object to have **multiple instances** of itself, then the object should be declared using the **Object Constructor Notation**.

```
function Vehicle(topSpeed, state) {  
  this.topSpeed = topSpeed;  
  this.state = state;  
}  
  
Vehicle.prototype.move = function (status) {  
  if (status === true) {  
    this.state = "moving";  
  } else this.state = "stopped";  
};  
  
myCar = new Vehicle(300);  
myCar.move(true);  
console.log(myCar.state); // prints "moving"  
  
yourCar = new Vehicle(250);  
yourCar.move(false);  
console.log(yourCar.state); // prints "stopped"
```

Try...Catch Statement in JavaScript

Table of Contents



Using the try...catch Statement

JavaScript supports the **try...catch** statement which can be used to catch and handle errors gracefully. It can contain the following **blocks**.



TRY

If an error occurs in the **try block**, code execution in the block ceases and the **catch block** is executed.



CATCH

The **catch block** receives an **object** with information about the error. Its value is only available in the **scope** of the catch block.



FINALLY

This block is **executed** after the try block is executed **regardless** of whether an exception was thrown or not.

Using the try...catch Statement

The try...catch statement uses the following syntax. It must have **at least** one **catch-block** or **finally-block** and therefore may use the following valid forms:

```
try {  
  
    // try block logic  
  
} catch (exception_object) {  
  
    // catch block logic  
  
} finally {  
  
    // finally block logic  
}
```

```
try {  
  
    // try block logic  
  
} catch (exception_object) {  
  
    // catch block logic  
}
```

```
try {  
  
    // try block logic  
  
} finally {  
  
    // finally block logic  
}
```

Using the try...catch Statement

Code that **may cause errors** should be placed inside a **try** block. When an error occurs, the **catch** block is executed instead of terminating the transaction.

```
// in this example, a function is used to update a record.  
// Should an error occur, the catch block executes and prints any details of the error  
function updateAmount(data) {  
  
    try {  
        lockRecord(data.id); // lock record  
        writeRecord(data); // update record  
    } catch (error) {  
        console.log('Error occurred when writing to record: ' + error);  
    } finally {  
        unlockRecord(); // unlock record whether an error was thrown or not  
    }  
}  
  
updateAmount({ id: 'a043i00000ApNUQAA3', amount: 10000, currency: 'usd' });
```

The **error** object in the catch block contains details of the error.

Learn More

-  [Testing Custom Controllers and Controller Extensions](#)
-  [Test Lightning Web Components](#)
-  [Testing Components](#)
-  [UI Test Automation Model](#)
-  [Trailhead - JavaScript Skills for Salesforce Developers](#)
-  [Trailhead - Modern JavaScript Development](#)

